

Programación de microcontroladores

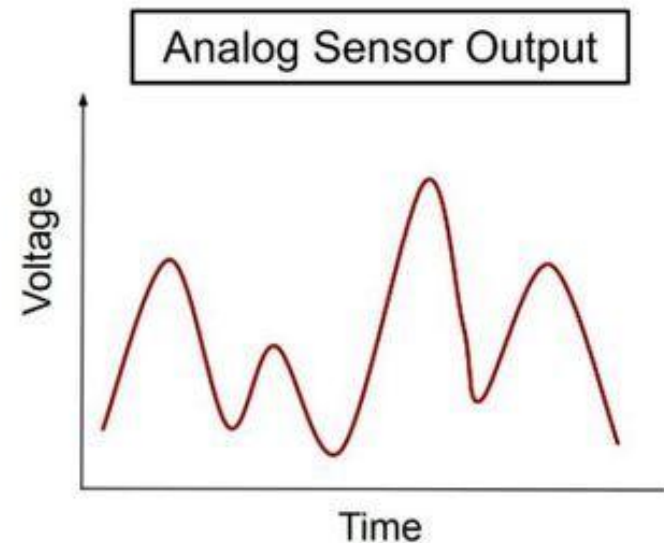
Entradas analógicas: ADC

Juan José Londoño, M.Sc.

Introducción

Los microcontroladores son sistemas digitales, pero muchos fenómenos físicos son **analógicos**.

Muchos sensores producen voltajes continuos, pero el microcontrolador solo entiende números binarios.

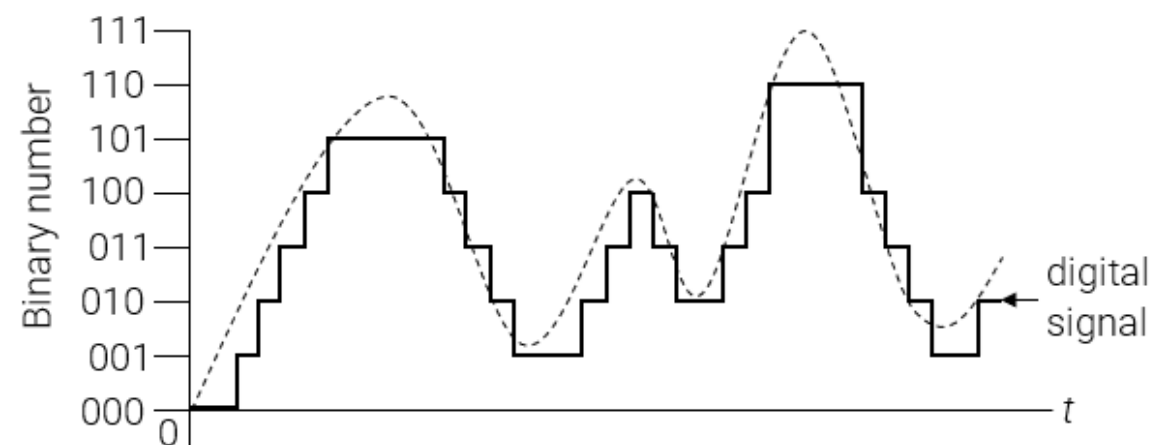
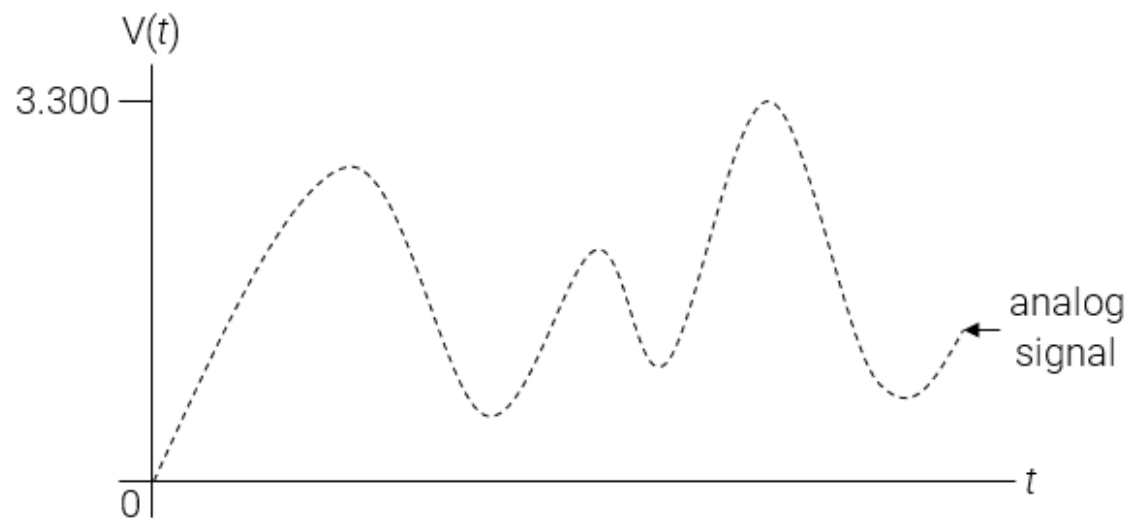


¿Qué es un ADC?

Un ADC convierte un voltaje analógico continuo en un número digital.

Un ADC (*Analog to Digital Converter*) es un circuito electrónico que:

- Mide un voltaje analógico.
- Lo **compara** internamente.
- Genera un número digital equivalente.



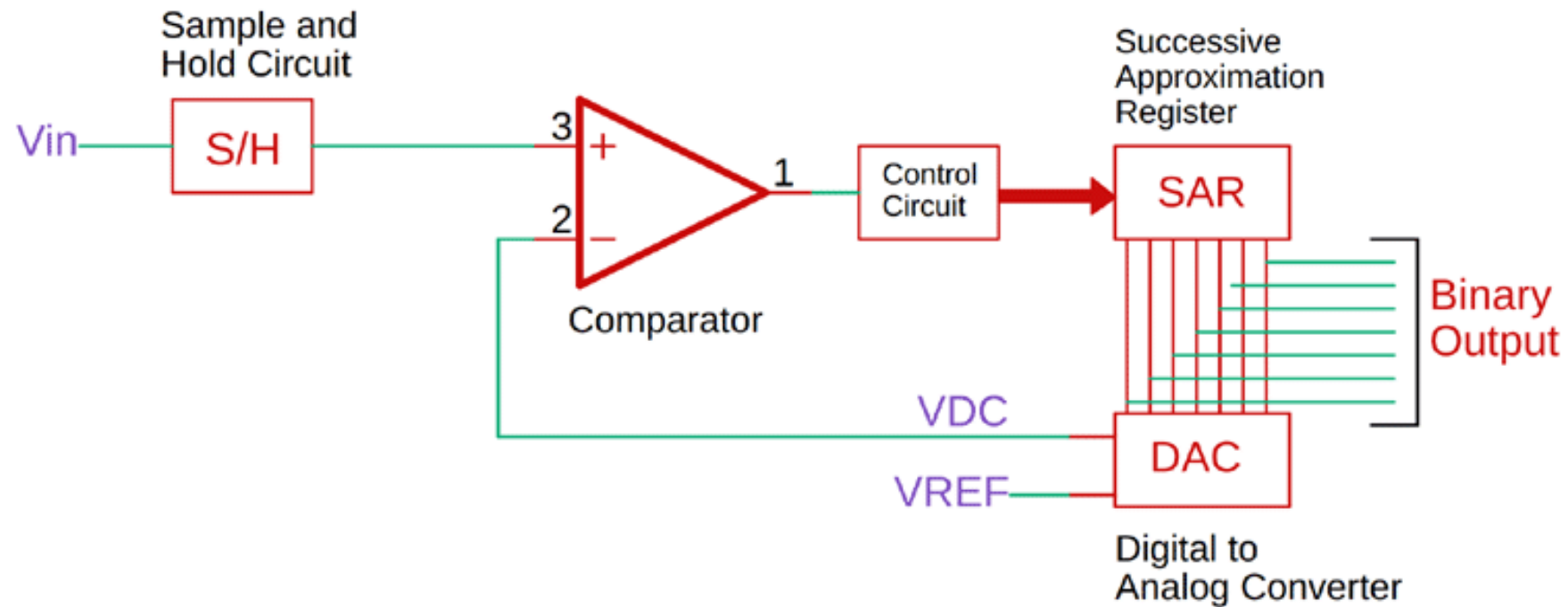
Rango de voltaje	Nivel
0 – 0.5 V	0
0.5 V – 1 V	1
1 V – 1.5 V	2
1.5 V – 2 V	3
2 V – 2.5 V	4

SAR ADC

Un SAR ADC (*Successive Approximation Register*) es un convertidor analógico-digital que encuentra el valor de un voltaje de entrada probando bits uno por uno, desde el más significativo hasta el menos significativo.

El ADC no conoce directamente el valor del voltaje de entrada. Entonces lo va “adivinando” paso a paso, comparándolo contra voltajes de prueba generados internamente.

SAR ADC



Funcionamiento del SAR ADC

Ejemplo para un ADC de 4 bits

Queremos convertir:

$$V_{in} = 2.4V$$

Paso 1

El registro SAR arranca probando el bit más significativo:

1000

Funcionamiento del SAR ADC

Para un rango de 0 a 3.3 V, el DAC genera el voltaje correspondiente a ese código de prueba:

El comparador arroja que: $2.4\text{ V} > 1.65\text{ V}$

Entonces el bit se mantiene:

1000

Ahora se sabe que V_{in} está entre 1.65 V y 3.3 V

Rango:

$$1111_2 = 3.3\text{ V}$$

$$1000_2 = 1.65\text{ V}$$

Funcionamiento del SAR ADC

Paso 2

Ahora el SAR prueba el siguiente bit.

1**1**00

Eso corresponde aproximadamente a:

$$1.65 \text{ V} + 0.825 \text{ V} = 2.475 \text{ V}$$

Funcionamiento del SAR ADC

Comparación: ¿ $2.4\text{ V} > 2.475\text{ V}$?

No se cumple, entonces, el bit del registro se pone en 0.

1000

Ahora se sabe que:

$$1.65\text{ V} < V_{in} < 2.475\text{ V}$$

Funcionamiento del SAR ADC

Paso 3

Ahora el SAR prueba el siguiente bit.

10**1**0

Eso corresponde aproximadamente a:

$$1.65V + 0.4125V = 2.0625V$$

Funcionamiento del SAR ADC

Comparación: ¿ $2.4V > 2.0625V$?

Sí se cumple, entonces, el bit del registro permanece en 1.

1010

Ahora se sabe que:

$$2.0625 V < V_{in} < 2.475 V$$

Funcionamiento del SAR ADC

Paso 4

Ahora el SAR prueba el bit menos significativo.

1011

Eso corresponde aproximadamente a:

$$2.0625V + 0.206V \approx 2.268V$$

Funcionamiento del SAR ADC

Comparación: ¿ $2.4V > 2.268V$?

Sí se cumple, entonces, el bit del registro permanece en 1.

1011

El registro final es **1011** (11 en decimal) y ese es el valor que entrega el ADC.

Ese es el valor digital que mejor representa **2.4 V** en un ADC de **4 bits** con **rango 0–3.3 V**.

Paso	Prueba	Voltaje DAC	Comparación	Resultado
1	1000	1.65 V	$2.4 > 1.65$	Bit = 1
2	1100	2.475 V	$2.4 < 2.475$	Bit = 0
3	1010	2.0625 V	$2.4 > 2.0625$	Bit = 1
4	1011	2.268 V	$2.4 > 2.268$	Bit = 1

Características del ADC

Resolución

La resolución indica cuántos niveles digitales diferentes puede representar el ADC. Se expresa en bits (N).

$$Niveles = 2^N$$

Esto significa que el rango de voltaje del ADC se divide en muchos niveles discretos. la resolución determina **qué tan fino** puede distinguir cambios de voltaje.

Características del ADC

Rango de entrada

Es el intervalo de voltajes que el ADC puede medir. Los sensores deben generar voltajes dentro del rango del ADC.

Voltaje de referencia (V_{ref})

El voltaje de referencia define cuál es el voltaje que corresponde al valor máximo del ADC. Es extremadamente importante porque determina **la escala de medición**.

Características del ADC

Tamaño del paso (LSB)

Representa el cambio mínimo de voltaje que el ADC puede detectar.

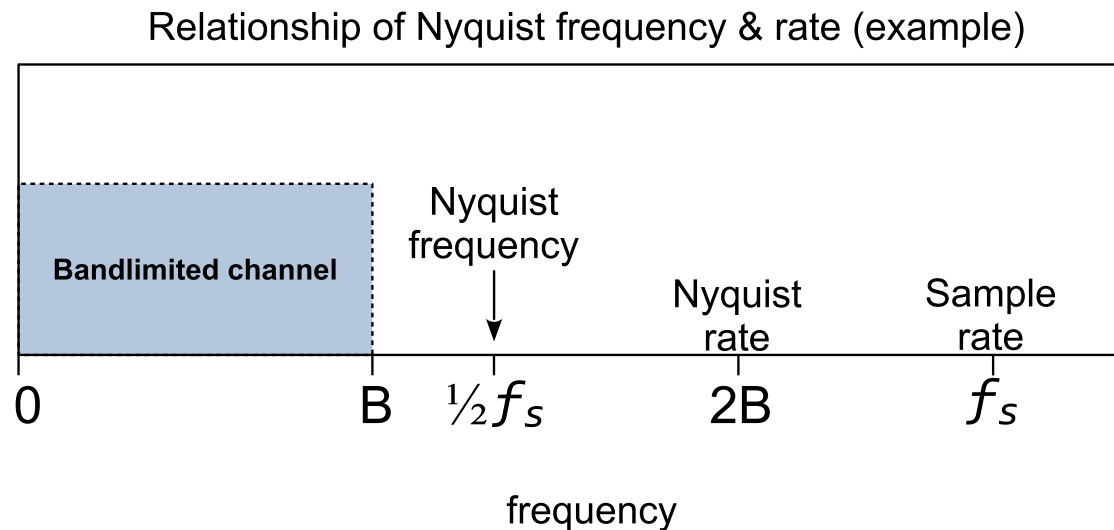
$$LSB = \frac{V_{ref}}{2^N}$$

Donde N es la resolución del ADC.

Frecuencia de muestreo

Es la cantidad de conversiones por segundo.

Se mide en muestras por segundo (SPS – *Sample Per Second*) o en Hz.



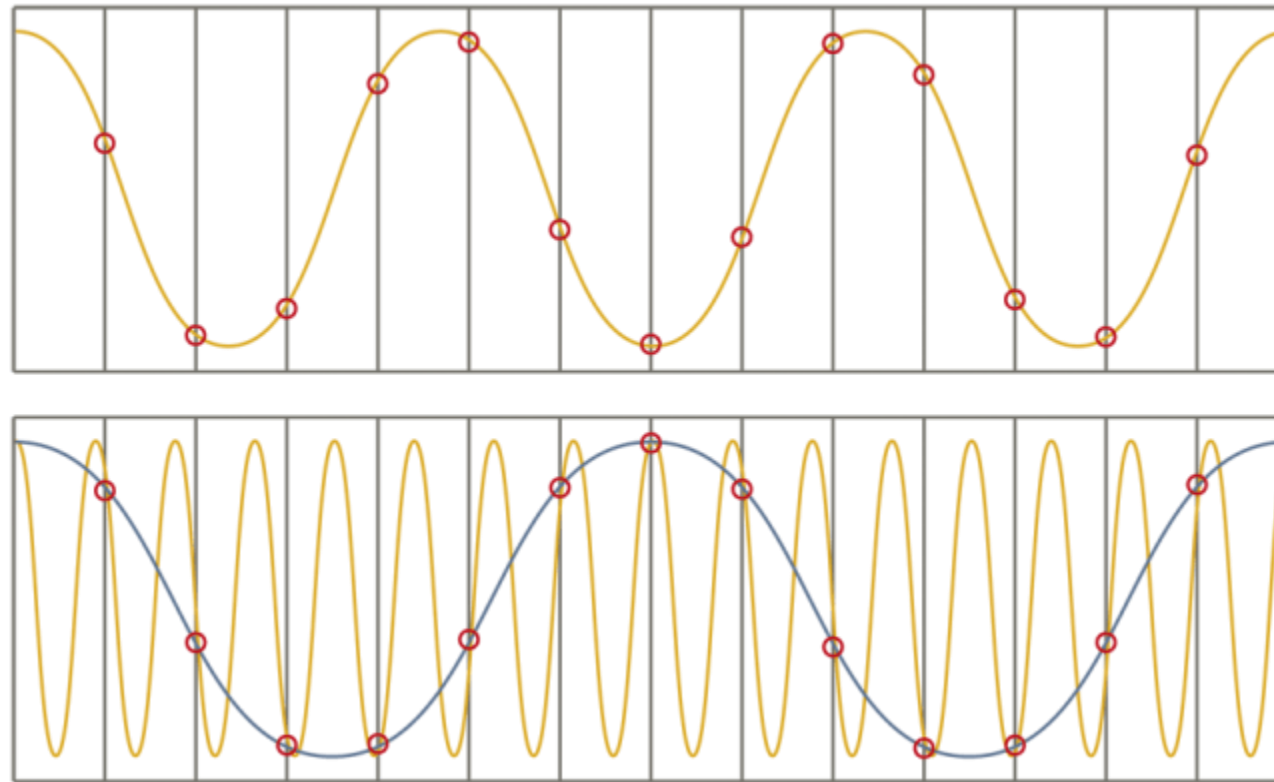
Aliasing

Un ADC no mide la señal continuamente, solo mide en instantes discretos.

Si la señal analógica cambia muy rápido, entonces el ADC no ve la señal real, incluso puede ver algo completamente diferente, este fenómeno se conoce como **aliasing**.

El aliasing ocurre cuando una señal rápida se interpreta como una señal más lenta.

Aliasing



Aliasing

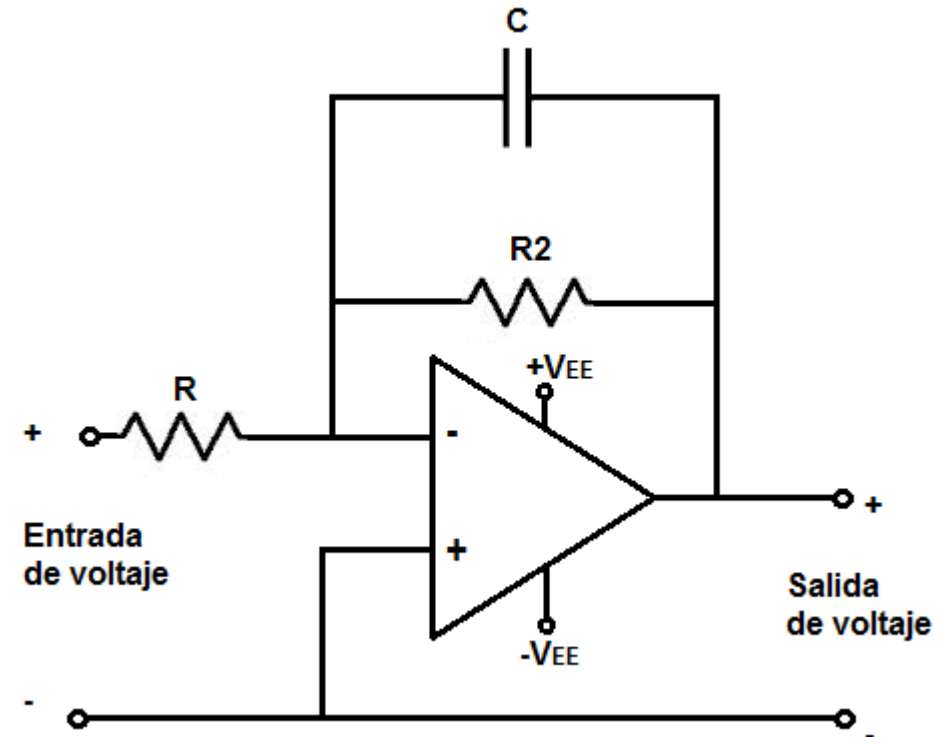
Usando el teorema de Nyquist se puede obtener un muestreo adecuado de la señal, el problema es que en la vida real las señales **nunca son perfectas**, siempre hay componentes de frecuencia más alta debido a **ruido** e **interferencias**.

Entonces, aunque el sensor sea lento, puede haber **frecuencias altas presentes** y si esas frecuencias llegan al ADC, pueden generar aliasing.

Filtro anti-aliasing

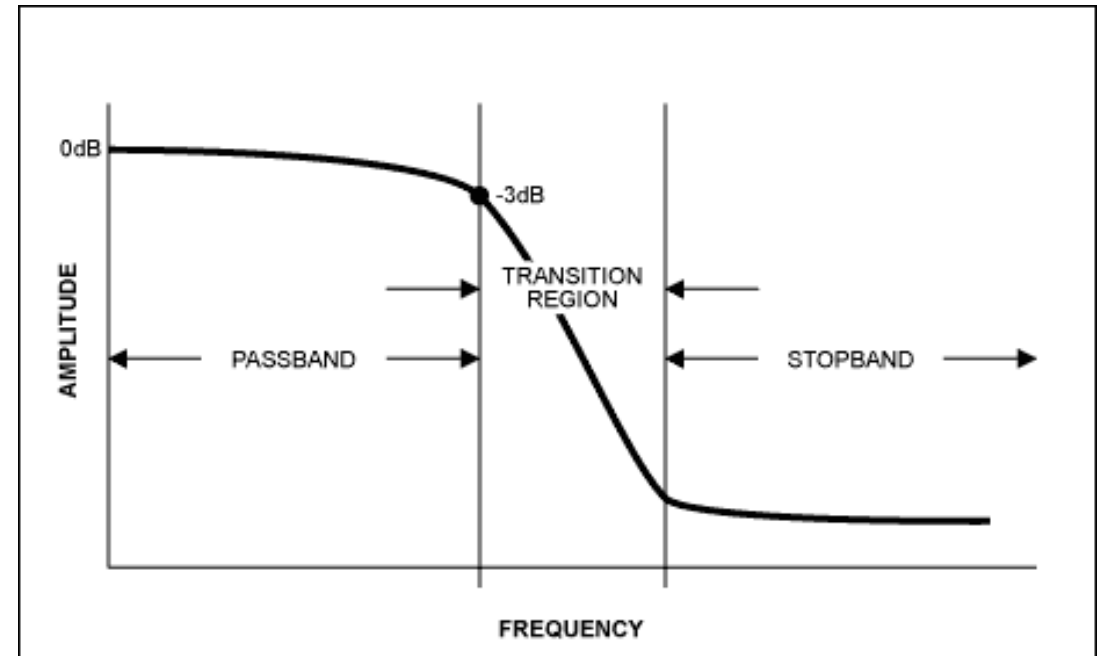
Es simplemente un filtro pasa-bajas analógico.

Su función es eliminar las frecuencias demasiado altas antes de que la señal llegue al ADC.



Filtro anti-aliasing

Si el ADC muestrea a 10 kHz, por ejemplo, la frecuencia máxima permitida debe ser de 5 kHz, por lo que el filtro debe diseñarse con un corte cercano a 4 – 5 kHz para eliminar componentes superiores.



ADC en el ESP32

El ESP32 tiene dos módulos ADC independientes:

- **ADC1.**
- **ADC2.**

Ambos son SAR ADC de **12 bits**.

Por lo que para un rango de 0–3.3 V tienen un LSB de 0.0008 V. Eso significa que el ADC solo puede distinguir cambios de aproximadamente **0.8 mV**.

ADC1 en el ESP32

Características:

- 8 canales.
- Puede usarse sin restricciones.
- Funciona incluso cuando el WiFi está activo.

Por eso suele ser el ADC recomendado.

Atenuación del ADC

El ADC permite configurar atenuación (*attenuation*), Esto cambia el rango de voltaje que puede medirse.

Atenuación	Rango aproximado
0 dB	0 – 1.1 V
2.5 dB	0 – 1.5 V
6 dB	0 – 2.2 V
12 dB	0 – 3.3 V

Modos de uso del ADC

Con la API de ESP-IDF , el ADC se trabaja principalmente en dos modos:

- Oneshot mode.
- Continuous mode.

El uso de cada modo depende de la aplicación y tiene diferentes características.

Modos de uso del ADC

Oneshot mode

Oneshot significa que el firmware le pide al ADC una **conversión puntual**, justo cuando se necesita.

Es un modo para conversiones *single, on-demand* y para muestrear un canal a la vez.

Este es el modo ideal cuando la señal es lenta o cuando no se necesita adquirir datos todo el tiempo.

Modos de uso del ADC

Continuous mode

Cuando el ADC debe muestrear de manera continua, automática y más rápida.

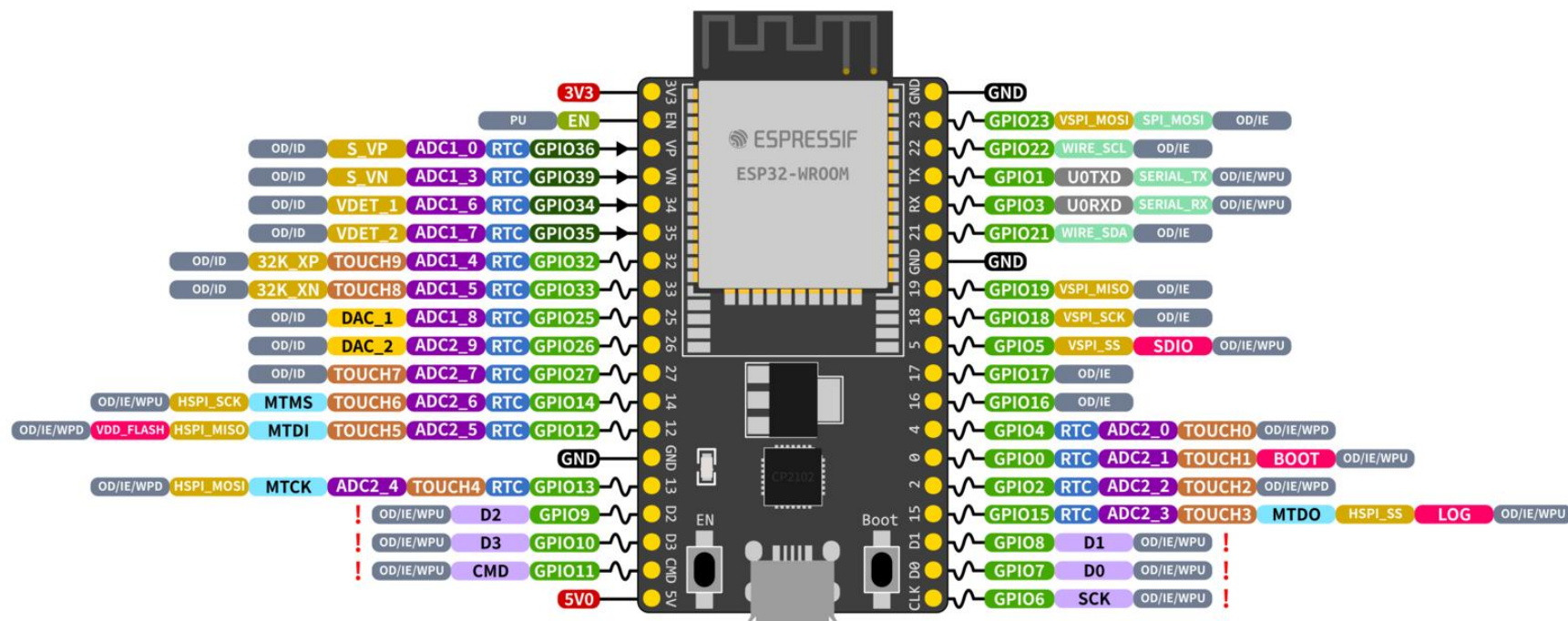
Es una forma de muestreo automático de uno o varios canales, transfiriendo resultados a memoria.

Se usa cuando se trabaja con señales más rápidas y se desean adquirir muchas muestras seguidas.

Calibración del ADC

La calibración sirve para mejorar la conversión desde el valor *raw* del ADC hacia un voltaje más cercano al real.

Espressif explica que el ADC del ESP32 trabaja con una referencia nominal de **1100 mV**, pero el valor real puede variar entre chips, y por eso existe el driver de calibración.



ESP32 Specs

32-bit Xtensa® dual-core @240MHz
 Wi-Fi IEEE 802.11 b/g/n 2.4GHz
 Bluetooth 4.2 BR/EDR and BLE
 520 KB SRAM (16 KB for cache)
 448 KB ROM
 34 GPIOs, 4x SPI, 3x UART, 2x I2C,
 2x I2S, RMT, LED PWM, 1 host SD/eMMC/SDIO,
 1 slave SDIO/SPI, TWAI®, 12-bit ADC, Ethernet



RTC: RTC Power Domain (VDD3P3_RTC)
GND: Ground
PWD: Power Rails (3V3 and 5V)
!: Pin Shared with the Flash Memory
 Can't be used as regular GPIO

GPIO STATE

- WPU**: Weak Pull-up (Internal)
- WPD**: Weak Pull-down (Internal)
- PU**: Pull-up (External)
- IE**: Input Enabled (After Reset)
- ID**: Input Disabled (After Reset)
- OE**: Output Enabled (After Reset)
- OD**: Output Disabled (After Reset)

Configuración del ADC



```
#include "freertos/FreeRTOS.h"  
#include "freertos/task.h"  
  
#include "esp_adc/adc_oneshot.h"
```

Handle del ADC



```
adc_one-shot_unit_handle_t adc1_handle;
```

Configuración del canal

```
adc_one_shot_unit_init_cfg_t init_config = {  
    .unit_id = ADC_UNIT_1,  
};  
adc_one_shot_new_unit(&init_config, &adc1_handle);  
  
adc_one_shot_chan_cfg_t chan_config = {  
    .bitwidth = ADC_BITWIDTH_DEFAULT,  
    .atten = ADC_ATTEN_DB_12,  
};  
adc_one_shot_config_channel(adc1_handle, ADC_CHANNEL_6,  
    &chan_config);
```

Captura del valor del ADC



```
int raw;  
adc_oneShot_read(adc1_handle, ADC_CHANNEL_6, &raw);
```

Frecuencia de muestreo

```
#define SAMPLE_PERIOD_US 1000

timer_init(TIMER_GROUP_0, TIMER_0, &timer_conf);
timer_set_counter_value(TIMER_GROUP_0, TIMER_0, 0);

uint64_t timer_value = 0;
int adc_raw;

while (1) {

    timer_get_counter_value(TIMER_GROUP_0, TIMER_0, &timer_value);

    if (timer_value >= SAMPLE_PERIOD_US) {

        adc_oneshot_read(adc1_handle, ADC_CHANNEL_6, &adc_raw);
        timer_set_counter_value(TIMER_GROUP_0, TIMER_0, 0);

    }
}
```

Calibración del ADC



```
#include "freertos/FreeRTOS.h"  
#include "freertos/task.h"  
  
#include "esp_adc/adc_cali.h"  
#include "esp_adc/adc_cali_scheme.h"
```

Handler de calibración



```
adc_cali_handle_t adc_cali_handle;
```


Configuración de calibración

```
adc_cali_line_fitting_config_t cali_config = {  
    .unit_id = ADC_UNIT_1,  
    .atten = ADC_ATTEN_DB_12,  
    .bitwidth = ADC_BITWIDTH_DEFAULT,  
};  
adc_cali_create_scheme_line_fitting(&cali_config, &adc_cali_handle)
```

Calibración de lectura

```
int raw;  
int voltage_mv;  
adc_oneshot_read(adc1_handle, ADC_CHANNEL_6, &raw);  
adc_cali_raw_to_voltage(adc_cali_handle, raw, &voltage_mv);
```